Árvores Binárias

Resumo cap 13-árvores

Livro:

Estrutura de Dados em C

Autor: Silvio do Lago Pereira

Árvores Binárias

- Para criar um árvore binária: (pg. 131)

```
#define fmt "%d "                          // formato de exibicao dos itens
typedef int Item;                          // tipo dos itens na arvore
typedef struct arv {                       // estrutura dos nos da arvore
   struct arv *esq;
   Item item;
   struct arv *dir;
} *Arv;                                    // tipo de ponteiro para arvore
```

Figura 13.3 | Definições para criação de árvore binária.

Criação da árvore binária (pg. 132)

```
Arv arv(Arv e, Item x, Arv d) {
   Arv n = malloc(sizeof(struct arv));
   n->esq  = e;
   n->item = x;
   n->dir  = d;
   return n;
}
```

Figura 13.4 | Função para criação de um nó de árvore binária.

arv(NULL,1,NULL) cria uma árvore de raiz com item igual a 1 e subárvores iguais a NULL.

Exemplo

```
Arv R = arv(arv(arv(NULL,
                    4,
                    NULL),
                2,
                arv(NULL,
                    5,
                    NULL)),
            1,
            arv(NULL,
                3,
                arv(NULL,
                    6,
                    NULL)));
```

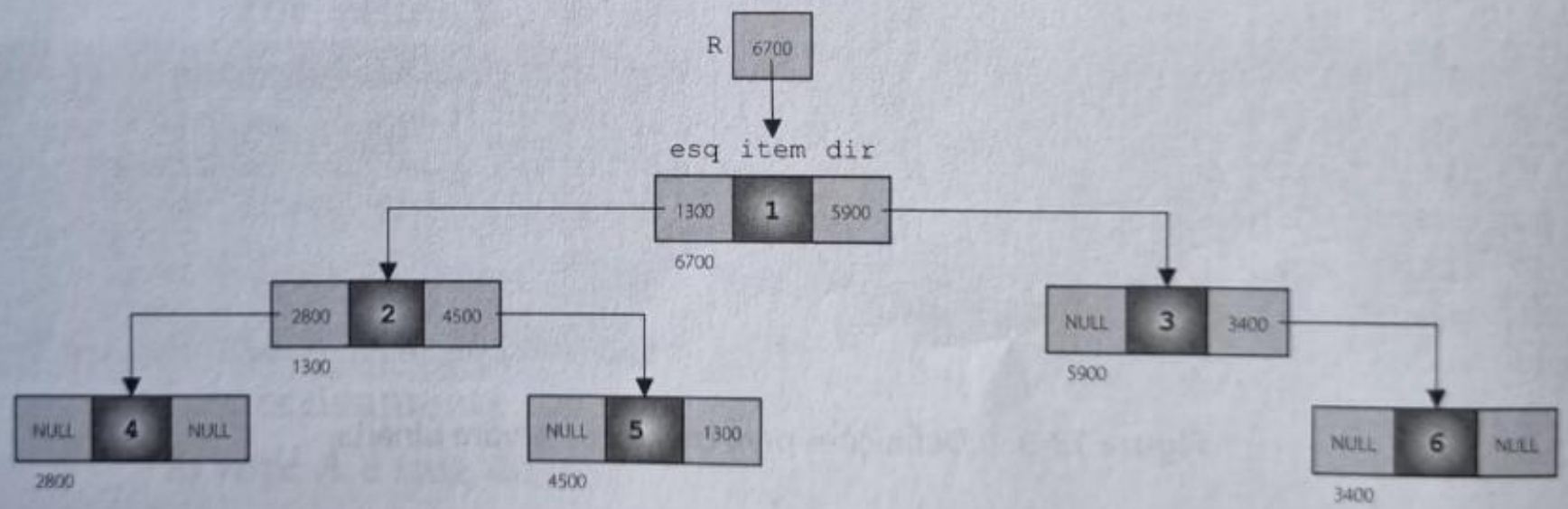


Figura 13.5 | Representação de árvore binária na memória.

Percurso em Árvore

- Um percurso é uma forma sistemática de visitar os nós de uma árvore.
- O **percurso em largura** visita os nós da esquerda para a direita.

Percurso em largura (extensão ou nível) Pg:133 (livro: ED em C. Silvio do Lago)

O *percurso em largura* visita os nós de uma árvore, em nível, da esquerda para a direita. Por exemplo, o percurso em largura visita os nós da árvore na Figura 13.5 nessa ordem: `1`, `2`, `3`, `4`, `5`, `6`. Para percorrer uma árvore em largura, criamos uma fila contendo apenas o endereço de seu nó raiz. A partir daí, enquanto a fila não estiver vazia: o endereço de um nó é removido da fila, o item nesse nó é exibido e os endereços de seus filhos são inseridos na fila. A função para percurso em largura, definida na Figura 13.6, usa o tipo `Fila`, definido no Capítulo 4.

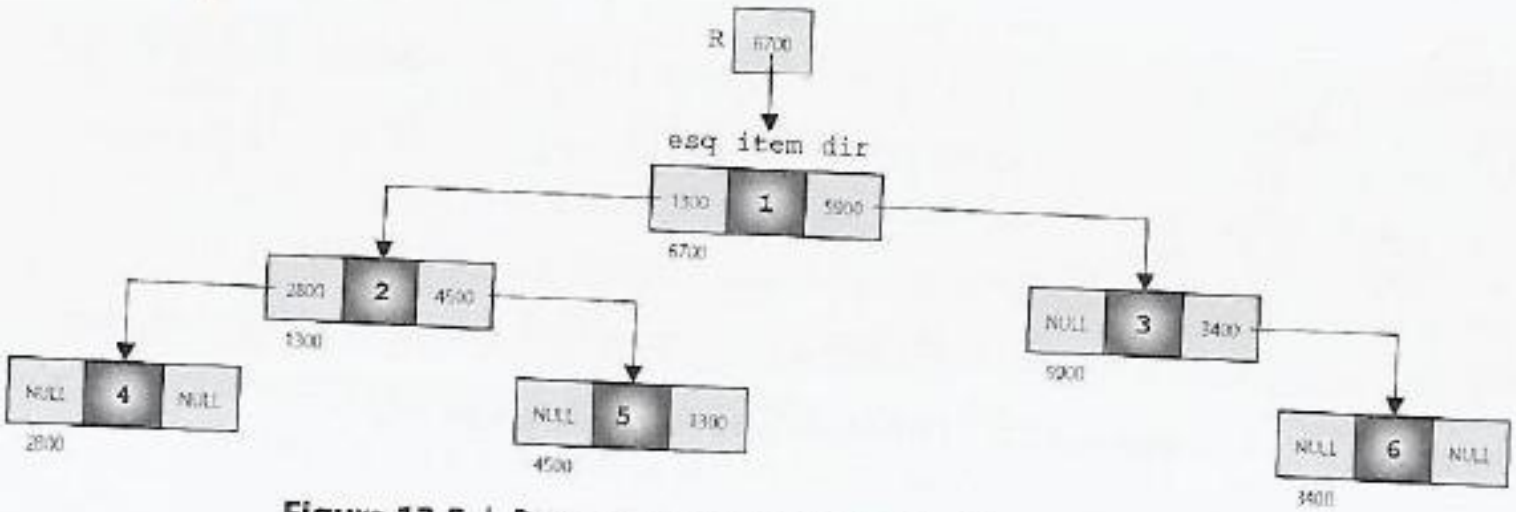


Figura 13.5 | Representação de árvore binária na memória.

Percurso em largura pag. 133

```
void emnivel(Arv A) {
   if( A==NULL ) return;
   Fila F = fila(MAX); // fila de Arv
   enfileira(A,F);
   while( !vaziaf(F) ) {
      Arv A = desenfileira(F);
      printf(fmt,A->item);
      if( A->esq != NULL ) enfileira(A->esq,F);
      if( A->dir != NULL ) enfileira(A->dir,F);
   }
}
```

Figura 13.6 | Função para percurso em largura (ou em nível).

- Para percorrer uma árvore binária de altura h>0, com todos os níveis completos, MAX deve ser definido como 2^h-1

FIGURA 6.10 Implementação do percurso em extensão (

```
template<class T>
void BST<T>::breadthFirst() {
    Queue<BSTNode<T>*> queue;
    BSTNode<T> *p = root;
    if (p != 0) {
        queue.enqueue(p);
        while (!queue.empty()) {
            p = queue.dequeue();
            visit(p);
            if (p->left != 0)
                queue.enqueue(p->left);
            if (p->right != 0)
                queue.enqueue(p->right);
        }
    }
}
```

Percurso em extensão do topo para baixo da esquerda pra direita (implementação similar em fig. 6.10 – pg 196 do livro ED e Algoritmos em C++. Adam Drozdek)

Implementação C++

```
void percusoExtensao(){
    queue<ArvoreNo<T>*> f;
    ArvoreNo<T> *p=root;
    if (p!=NULL){
        f.push(p);
        while (!f.empty()){
            p=f.front();
            visit(p);
            f.pop();
            if (p->left!=0)
                f.push(p->left);
            if (p->right!=0)
                f.push(p->right);
        }   }
}
```

Sobre o percurso em largura (extensão ou nível):

(a) O tamanho MAX da fila depende da árvore percorrida

(b) Considerando i o nível da árvore e lembrando que uma **<u>árvore binária com todos os níveis completos</u>** possui 2^{i-1} nós em cada nível da árvore:

- Se TODOS os nós em TODOS os níveis tiverem 2 filhos, então haverá **1 = 2°** nós no **nível 1**.
- **2 = 2^1** nós no **nível 2**.
- **4 = 2^2** nós no **nível 3...**

O maior nível vai determinar a altura da árvore (h). Logo o tamanho da fila depende diretamente da altura da árvore percorrida. Para percorrer uma árvore binária de altura h>0, como todos os níveis completos, o tamanho máximo da fila será de 2^h - 1

Percurso em Profundidade (pg. 133)

Um *percurso em profundidade* pode ser de três tipos básicos:

- **Em-ordem**: percorre a subárvore esquerda, depois visita a raiz da árvore e, finalmente, percorre a subárvore direita. Por exemplo, o percurso em-ordem visita os nós da árvore na Figura 13.5 nessa ordem: 4, 2, 5, 1, 3, 6.
- **Pré-ordem**: visita a raiz da árvore, depois percorre a subárvore esquerda e, finalmente, percorre a subárvore direita. Por exemplo, o percurso pré-ordem visita os nós da árvore na Figura 13.5 nessa ordem: 1, 2, 4, 5, 3, 6.
- **Pós-ordem**: percorre a subárvore esquerda, depois percorre a subárvore direita e, finalmente, visita a raiz da árvore. Por exemplo, o percurso pós-ordem visita os nós da árvore na Figura 13.5 nessa ordem: 4, 5, 2, 6, 3, 1.

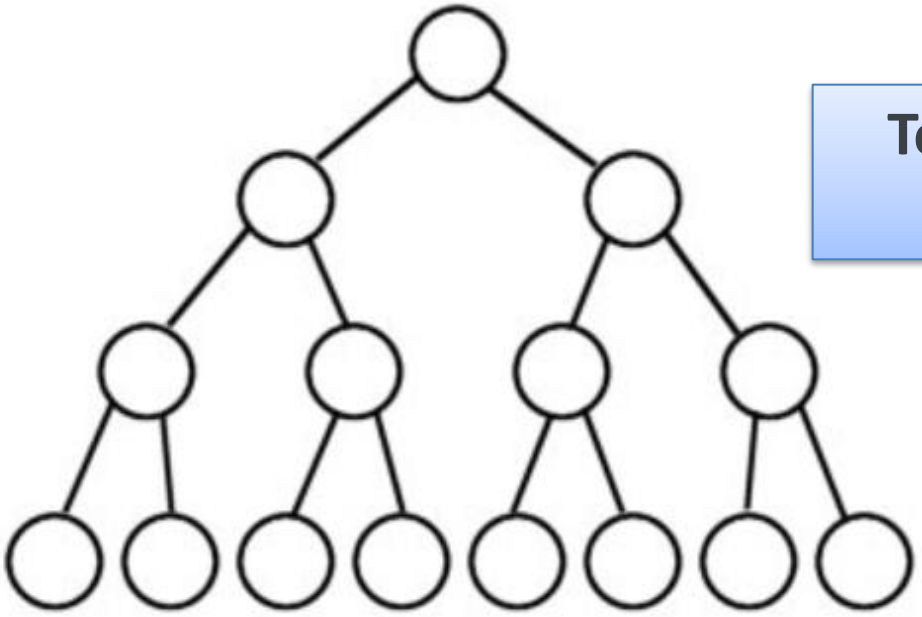
- Percurso em largura x percurso em profundidade (pg. 133 cap. 13 - Árvores):

A vantagem dos percursos em profundidade é que eles podem ser facilmente implementados de forma recursiva. Além disso, enquanto o espaço de memória usado pelo percurso em largura cresce *exponencialmente* em função da altura da árvore percorrida, o espaço de memória usado pelos percursos em profundidade cresce apenas *linearmente* em função dessa altura.

Árvore estritamente binária: Cada nó possui exatamente 0 ou 2 filhos.
Árvore binária completa: Nós com menos de 2 filhos ficam no último ou no penúltimo nível da árvore.
Árvore binária cheia: Nós com menos de 2 filhos ficam no último nível da árvore.

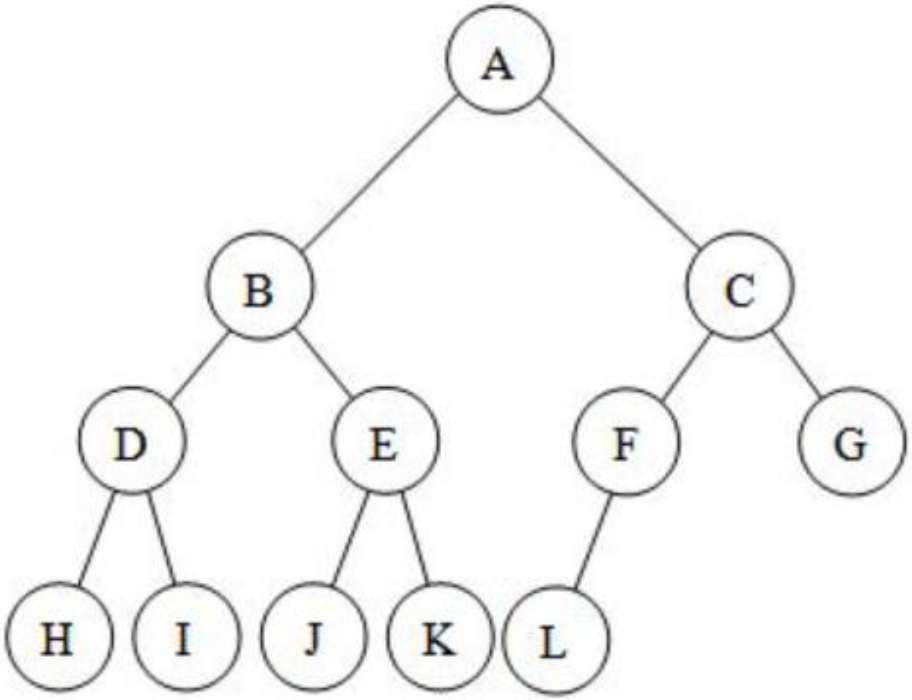
Árvore binária cheia : é uma árvore em que cada nó na árvore tem 0 ou 2 nós filhos(ou folhas).

Full Binary Tree



Toda árvore cheia é completa e estritamente binária.

Árvore binária completa : em uma árvore binária completa todos os níveis, exceto possivelmente o último , está completamente cheia, e todos os nós no último nível são, tanto à esquerda quanto possível.



Na **árvore binária completa**, a exigência de que os nós estejam alinhados à esquerda no último nível tem a ver com sua definição e eficiência estrutural. Essa característica garante que:

1. **Organização Sistemática**: Alinhar os nós à esquerda evita espaços "vazios" no meio da estrutura, garantindo que todos os níveis sejam preenchidos sequencialmente e de forma ordenada. Isso facilita operações como inserção e remoção, principalmente quando a árvore é representada em forma de array.
2. **Eficiência em Estruturas de Dados**: Muitas implementações de árvores binárias completas, como **heaps binários**, utilizam arrays para armazenamento. Posicionar os nós à esquerda tanto quanto possível permite que os índices do array sejam calculados diretamente para os filhos e o pai de cada nó, sem criar lacunas inúteis.
3. **Redução de Complexidade Computacional**: Quando os nós estão alinhados à esquerda, é mais fácil determinar o tamanho e o comportamento da árvore sem precisar verificar a presença de espaços vazios. Isso otimiza o processamento em algoritmos que utilizam essa estrutura.

Aqui estão alguns exemplos de árvores binárias representando **heaps binários**:

1. Heap Máximo (Max-Heap)

Um heap máximo é uma árvore binária em que o valor de cada nó pai é **maior ou igual** aos valores de seus filhos.

Exemplo:

```
        50
      /   \
    30     40
   / \     /
  10  20 15
```

Nesta árvore, cada nó pai (como 50, 30 e 40) tem um valor maior do que seus filhos diretos.

2. Heap Mínimo (Min-Heap)

Um heap mínimo é uma árvore binária em que o valor de cada nó pai é **menor ou igual** aos valores de seus filhos.

Exemplo:

```
        10
       /  \
     20    15
    / \    /
   40  30 50
```

Aqui, cada nó pai (como 10, 20 e 15) tem um valor menor do que seus filhos diretos.

Representação em Array:

Heaps binários geralmente são representados em arrays para facilitar operações. Aqui está como os exemplos acima seriam representados:

- **Max-Heap Array**: [50, 30, 40, 10, 20, 15]
- **Min-Heap Array**: [10, 20, 15, 40, 30, 50]

As relações de pai e filho no array seguem:

- Índice do pai: (i)
- Filho esquerdo: (2i + 1)
- Filho direito: (2i + 2)

Os heaps binários têm diversas aplicações práticas no mundo real devido à sua eficiência em operações específicas. Aqui estão alguns exemplos:

1. Gerenciamento de Tarefas e Prioridades (Fila de Prioridade)

- Um heap é usado para implementar **filas de prioridade**, onde cada tarefa possui um "peso" ou "importância".
- Por exemplo:
  - Em sistemas operacionais, heaps ajudam a gerenciar processos para que os mais importantes (com maior prioridade) sejam executados primeiro.
  - Em aplicações como planejamento de voos, o heap organiza voos atrasados com base na severidade do atraso.

Algoritmo de Ordenação (Heap Sort)

Heaps binários são usados no **Heap Sort**, um método eficiente de ordenação que tem complexidade (O(n \log n)).

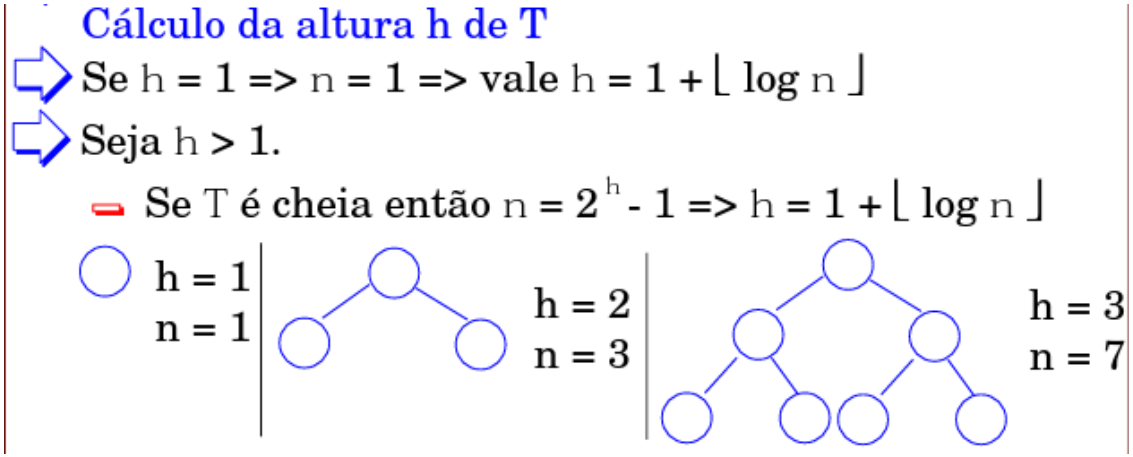
O heap máximo é construído, e o maior elemento (raiz) é removido iterativamente para formar uma lista ordenada. Isso é útil em situações onde grandes volumes de dados precisam ser organizados rapidamente.

Jogos e Inteligência Artificial

Heaps podem ser usados para gerenciar prioridades em jogos, como eventos ou movimentos. Em jogos de estratégia, heaps binários ajudam na execução das ações mais críticas primeiro.

ATIVIDADE

1. Faça um algoritmo que verifique a quantidade de nós em uma árvore binária.
2. Faça um algoritmo que calcule a altura (h) de uma árvore binária.
3. Faça um algoritmo que verifique se a árvore binária é completa ou não. Considere que a árvore binária completa possui uma altura h mínima (1 + log n):



4. Faça um algoritmo que verifique se a árvore binária é cheia.

REMOÇÃO DE UM ITEM MÁXIMO EM ÁRVORE BINÁRIA DE BUSCA (PG. 137)

A função que remove um item máximo de uma árvore de busca binária é definida na Figura 13.15. Note que o nó que guarda um item máximo da árvore de busca binária *não* pode ter filho à direita (senão, ele não seria máximo), mas ele pode ter um filho à esquerda (Figura 13.14b). Portanto, quando esse nó é removido, o ponteiro que o apontava deve passar a apontar seu filho à esquerda (caso ele não tenha filho à esquerda, o ponteiro que o apontava ficará nulo).

```
Item remmax(Arv *A) {
    if( *A == NULL ) abort();
    while( (*A)->dir != NULL ) A = &(*A)->dir;
    Arv n = *A;
    Item x = n->item;
    *A = n->esq;
    free(n);
    return x;
}
```

Figura 13.15 | Função que remove e devolve um item máximo de uma árvore de busca binária.

REMOÇÃO DE UM ITEM X QUE ESTÁ NA RAIZ DA ÁRVORE BINÁRIA DE BUSCA (PG. 137)

Agora, vamos considerar a remoção de um item `x`, que está na *raiz* da árvore de busca binária `A`. Então, há três casos possíveis:

- Se o nó apontado por `A` *não tem filhos*, então ele deve ser desalocado e o ponteiro `A` deve ser anulado (isto é, a árvore fica vazia).
- Se o nó apontado por `A` *tem um único filho*, então ele deve ser desalocado e o ponteiro `A` deve passar a apontar esse filho.
- Se o nó apontado por `A` *tem dois filhos*, então o item nesse nó deve ser substituído por um item máximo removido de sua subárvore esquerda.

REMOÇÃO DE UM ITEM X QUE ESTÁ NA RAIZ DA ÁRVORE BINÁRIA DE BUSCA (PG. 138)

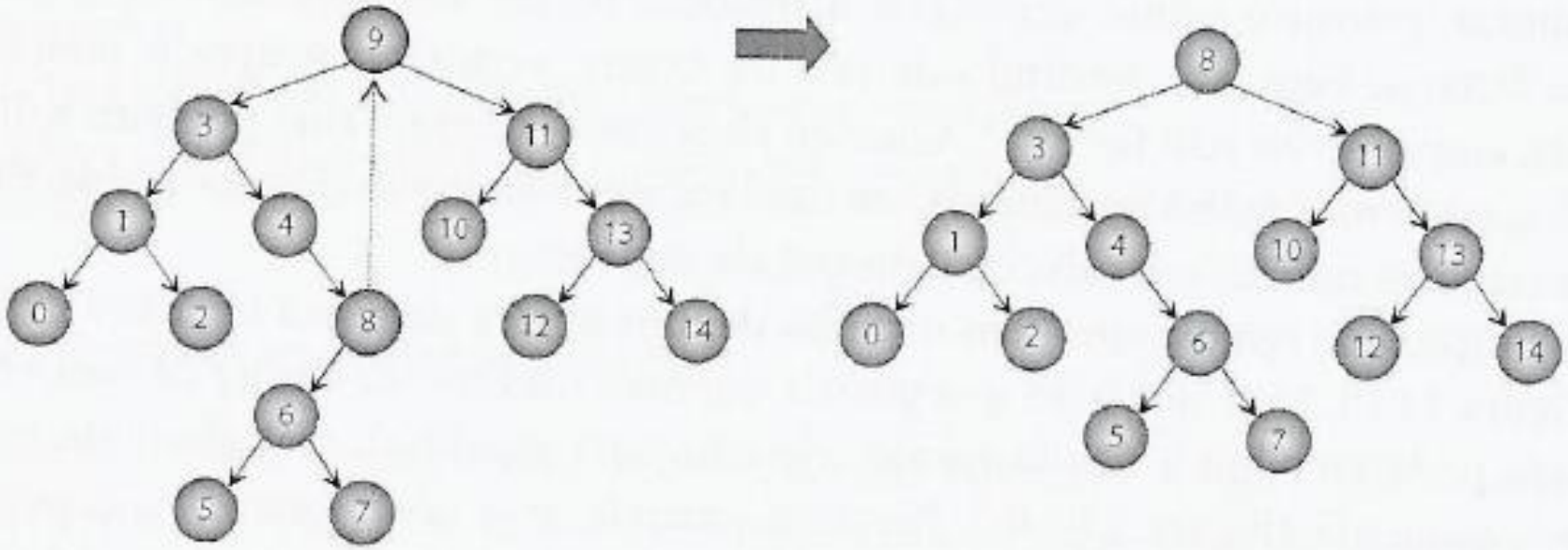


Figura 13.16 | Remoção de um nó com dois filhos.

REMOÇÃO DE UM ITEM X QUE ESTÁ NA RAIZ DA ÁRVORE BINÁRIA DE BUSCA (PG. 138)

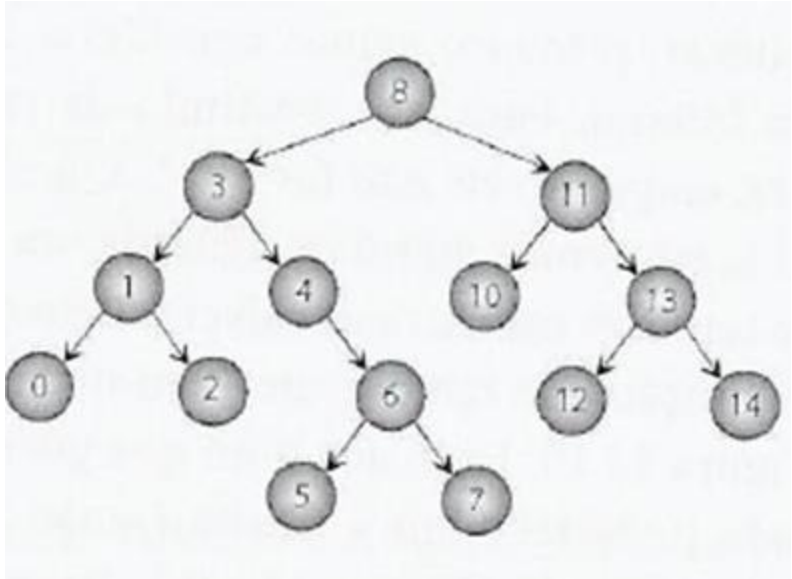
Para generalizar essa ideia, precisamos considerar que o item a ser removido pode estar em qualquer nó da árvore, não apenas na raiz. Nesse caso, antes de removê-lo, precisamos encontrá-lo na árvore. Para isso, basta adaptar a lógica da função de busca em árvore (Figura 13.13). A função que remove um item x qualquer de uma árvore de busca binária A é definida na Figura 13.17.

```
void rem(Item x, Arv *A) {
   if( *A == NULL ) return;
   if(x == (*A)->item ) {
      Arv n = *A;
      if( n->esq == NULL ) *A = n->dir;
      else if ( n->dir == NULL ) *A = n->esq;
      else n->item = remmax(&n->esq);
      if( n != *A ) free(n);
   }
   else if( x <= (*A)->item ) rem(x,&(*A)->esq);
   else rem(x,&(*A)->dir);
}
```

Figura 13.17 | Função para remoção em árvore de busca binária.

ATIVIDADE

1. Considerando a árvore abaixo:



1. Realize o algoritmo de remoção por fusão para apagar o item 8.
2. Realize o algoritmo de remoção apresentado na Fig. 14.17 do livro para apagar o item 8.
3. Compare as árvores resultantes em 1 e 2. Quais os níveis das árvores? São iguais ou diferentes?

ATIVIDADE (pg. 138 e 139)

Exercícios

13.1 Crie o arquivo `arv.h`, com os tipos e funções para árvores definidos nesse capítulo (exceto a função `emnivel()`, que depende do tipo `Fila`), e use esse arquivo num programa que cria e exibe a árvore da Figura 13.5.

13.2 Crie a função `nos(A)`, que devolve o total de *nós* na árvore binária `A`.

13.3 Crie a função `folhas(A)`, que devolve o total de *folhas* na árvore binária `A`.

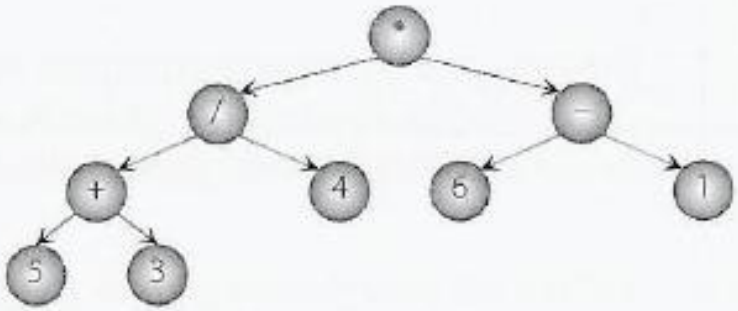
ATIVIDADE (pg. 138 e 139)

13.5 Crie a função `tem(A,x)`, que informa se a árvore binária `A` tem o item `x`.

13.6 Uma árvore `A` é *estritamente binária* se cada nó em `A` é uma folha ou tem dois filhos. Crie a função `eb(A)`, que informa a árvore `A` é estritamente binária.

13.7 Duas árvores binárias `A` e `B` são *iguais* se elas têm a mesma forma e os mesmos itens. Crie a função `igual(A,B)`, que informa se `A` é igual a `B`.

13.8 Uma expressão aritmética pode ser representada por uma árvore binária cuja raiz é uma operação e cujas subárvores são operandos. Por exemplo, a expressão `((5+3)/4)*(6-1)` pode ser representada como na figura a seguir. Crie a função `valor(A)`, que avalia uma expressão aritmética representada por uma árvore binária `A` (cujos nós guardam números inteiros).



13.9 Crie a função `exibe_dec(A)`, que exibe os itens de uma árvore de busca binária em ordem decrescente.